

Packaging a Custom UI

Information last updated 22 sept 2025

SME	Jan Rusch
EPIC	CIC Custom User Interface and Custom Development

1 Instructions

1.1 Custom UI Packaging Process Flow

1.2 Prerequisites

1.3 Step 1: Prepare for Packaging

1.3.1 1.1 Test Your Application

1.3.2 1.2 Commit Your Changes

1.4 Step 3: Build and Package Your UI

1.4.1 3.1 Run the Pack-Build Command

1.4.2 Deep Dive: What Happens During npm run build workspace-hxp

1.4.2.1 3.1.1 Pre-Build Phase

1.4.2.2 3.1.2 Angular Build Process

1.4.2.3 3.1.3 Build Output Structure

1.4.2.4 3.1.4 Post-Build Phase

1.4.2.5 3.1.5 Build Configuration Details

1.4.2.6 3.1.6 Build Performance Optimizations

1.4.2.7 3.1.7 Build Output Analysis

1.4.2.8 3.1.10 Build Output Validation

1.4.3 3.2 Verify the Build

1.5 Step 4: Deploy in Studio Modeling

1.5.1 4.1 Navigate to Your Custom UI

1.5.2 4.2 Upload the Package

1.5.3 4.3 Release and Deploy

1.6 Step 5: Test Your Deployed UI

1.6.1 5.1 Access Your Custom UI

1.6.2 5.2 Verify Everything Works

1.7 Common Issues and Solutions

1.7.1 Build Fails with File Copy Errors

1.7.2 Upload Fails

1.7.3 UI Doesn't Load After Deployment

1.7.4 Missing Assets

1.8 Best Practices

1.8.1 Before Packaging

1.8.2 During Packaging

1.8.3 After Deployment

1.9 Advanced Packaging Options

1.9.1 Environment-Specific Builds

1.9.2 Custom Build Scripts

1.10 Next Steps

1.11 Additional Resources

1.12 Questions?

2 Related articles

Instructions

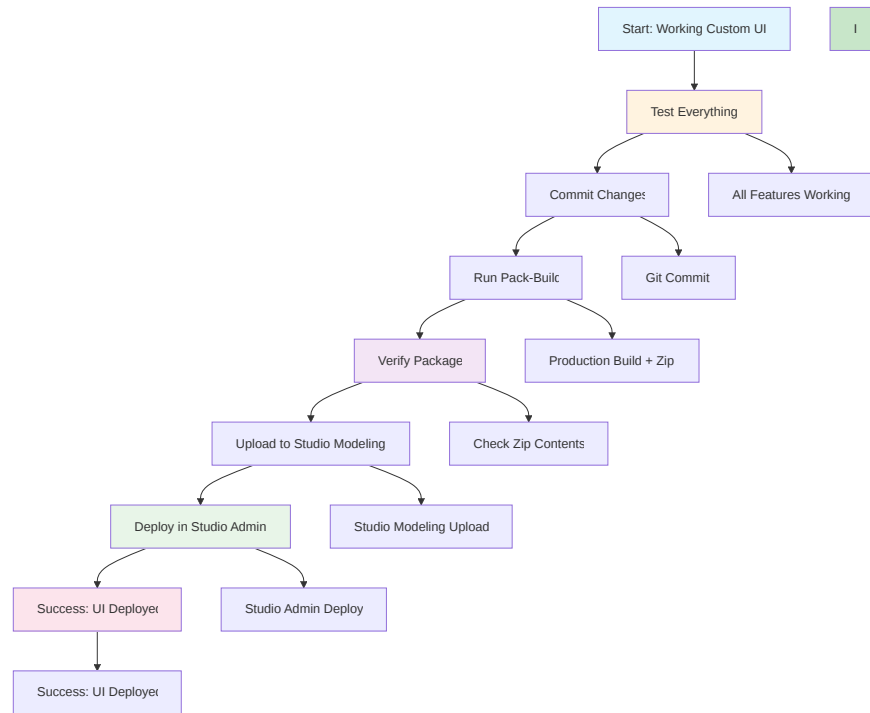
Hey ! Welcome back. I'm going to show you how to package and deploy your custom UI so it can be used in your Hyland Experience environment.

What you'll learn today:

- How to build your custom UI for production
- How to package it into a deployable zip file
- How to upload and deploy it in Studio Modeling
- Common issues and troubleshooting tips

Let's get your custom UI deployed!

Custom UI Packaging Process Flow



Prerequisites

Before we start, make sure you have:

- **A completed custom UI** with all your changes
- **All development work finished** and tested locally
- **Access to Studio Modeling** for deployment
- **A working Hyland Experience environment**

Step 1: Prepare for Packaging

1.1 Test Your Application

Before packaging, make sure everything works:

```
1 # Start your development server
2 npm start workspace-hxp
3
4 # Test all functionality
5 # - Navigation works correctly
6 # - All pages load properly
7 # - Forms and widgets function as expected
8 # - No console errors
```

1.2 Commit Your Changes

Make sure all your changes are committed to version control:

```
1 git add .
2 git commit -m "Ready for packaging - custom UI complete"
```

Step 3: Build and Package Your UI

3.1 Run the Pack-Build Command

```
1 npm run pack-build workspace-hxp
```

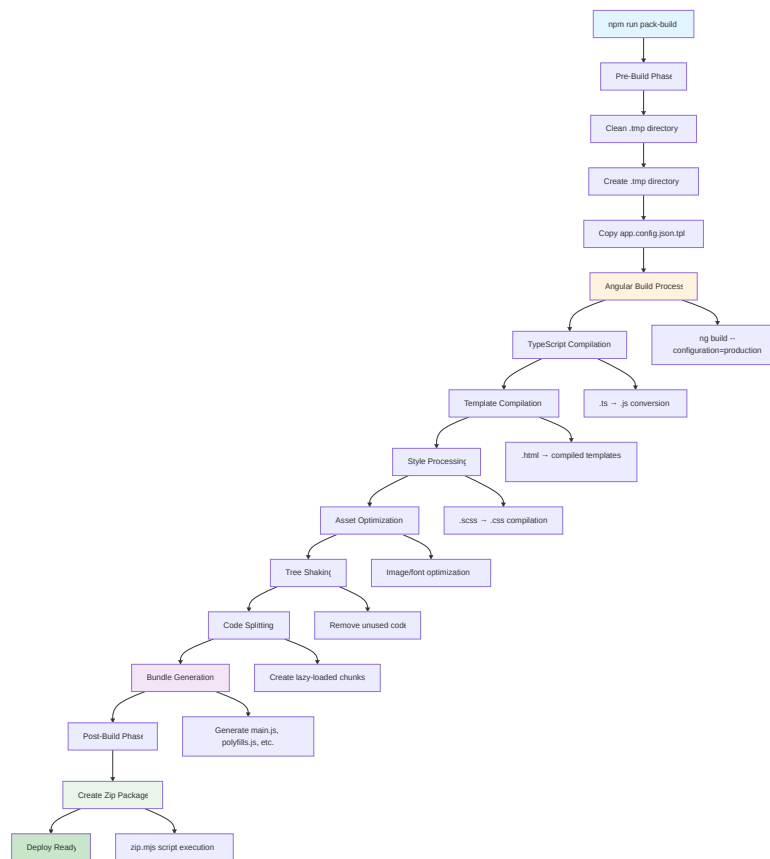
```
NX Successfully ran target build for project workspace-hxp and 1 task it depends on
Nx read the output from the cache instead of running the command for 1 out of 2 tasks.
Creating dist/workspace-hxp.zip from dist/workspace-hxp
7734849 total bytes
Finished zipping
NX Successfully ran target pack-build for project workspace-hxp (3s)
```

What this command does:

- Builds your application for production
- Optimizes the code for performance
- Creates a zip file with all necessary assets
- Places the zip file in the `/dist` folder

🔍 Deep Dive: What Happens During `npm run build workspace-hxp`

Let's break down exactly what happens when you run the build command:



3.1.1 Pre-Build Phase

```
1 # 1. Clean temporary directories
2 if exist .tmp rd /s /q .tmp
3
4 # 2. Create temporary directory
5 mkdir .tmp
6
7 # 3. Copy template configuration
8 copy .tmp\\app.config.json.tpl .tmp\\app.config.json
```

3.1.2 Angular Build Process

```
1 # 4. Run Angular production build
2 ng build workspace-hxp --configuration=production
```

What Angular build does:

- **TypeScript Compilation:** Converts `.ts` files to JavaScript
- **Template Compilation:** Processes `.html` templates
- **Style Processing:** Compiles SCSS to CSS
- **Asset Optimization:** Minifies and optimizes images, fonts, etc.
- **Tree Shaking:** Removes unused code
- **Code Splitting:** Creates optimized chunks for lazy loading
- **Source Maps:** Generates debugging maps (optional in production)

3.1.3 Build Output Structure

```
1 dist/workspace-hxp/
2   ├── index.html           # Main HTML file
3   ├── main.[hash].js       # Main application bundle
4   ├── polyfills.[hash].js  # Browser compatibility polyfills
5   ├── runtime.[hash].js    # Angular runtime
6   ├── styles.[hash].css    # Compiled styles
7   ├── assets/              # Static assets
8     ├── images/
9     ├── fonts/
10    ├── i18n/
11    ├── [chunk].[hash].js   # Lazy-loaded feature chunks
12    └── app.config.json     # Runtime configuration
```

3.1.4 Post-Build Phase

```
1 # 5. Create deployment package
2 tools/scripts/zip.mjs --source dist/workspace-hxp --target dist/workspace-hxp.zip
```

Zip creation process:

- **Collects all files** from the build output
- **Preserves directory structure** for proper deployment
- **Compresses assets** for faster upload/download
- **Creates deployment-ready package**

3.1.5 Build Configuration Details

The build uses these key configurations:

Angular Configuration (`angular.json`):

```
1 {
2   "build": {
3     "builder": "@angular-devkit/build-angular:browser",
4     "options": {
5       "outputPath": "dist/workspace-hxp",
6       "index": "src/index.html",
7       "main": "src/main.ts",
8       "polyfills": "src/polyfills.ts",
9       "tsConfig": "tsconfig.app.json",
10      "assets": ["src/assets"],
11      "styles": ["src/styles.scss"],
12      "scripts": []
13    },
14    "configurations": {
15      "production": {
16        "budgets": [
17          {
18            "type": "initial",
19            "maximumWarning": "2mb",
20            "maximumError": "5mb"
21          }
22        ],
23        "outputHashing": "all",
24        "sourceMap": false,
25        "namedChunks": false,
26        "extractLicenses": true,
```

```

27     "vendorChunk": false,
28     "buildOptimizer": true
29   }
30 }
31 }
32 }

```

TypeScript Configuration (tsconfig.app.json):

```

1 {
2   "extends": "../tsconfig.json",
3   "compilerOptions": {
4     "outDir": "../dist/out-tsc",
5     "types": []
6   },
7   "files": [
8     "src/main.ts",
9     "src/polyfills.ts"
10  ],
11  "include": [
12    "src/**/*.d.ts"
13  ]
14 }

```

3.1.6 Build Performance Optimizations

What gets optimized:

- **JavaScript Minification:** Removes whitespace, shortens variable names
- **CSS Minification:** Compresses stylesheets
- **Image Optimization:** Compresses images without quality loss
- **Bundle Splitting:** Separates vendor code from application code
- **Lazy Loading:** Creates separate chunks for feature modules
- **Tree Shaking:** Removes dead code
- **AOT Compilation:** Pre-compiles templates for faster runtime

3.1.7 Build Output Analysis

File Size Breakdown:

- **main.[hash].js:** ~500KB-2MB (your application code)
- **polyfills.[hash].js:** ~50-100KB (browser compatibility)
- **runtime.[hash].js:** ~10-20KB (Angular runtime)
- **styles.[hash].css:** ~50-200KB (compiled styles)
- **vendor.[hash].js:** ~500KB-1MB (third-party libraries)

3.1.10 Build Output Validation

What to check after build:

```

1 # 1. Verify zip file was created
2 ls -la dist/workspace-hxp.zip
3
4 # 2. Check zip contents
5 unzip -l dist/workspace-hxp.zip
6
7 # 3. Verify file sizes are reasonable
8 du -h dist/workspace-hxp/*
9
10 # 4. Check for missing assets
11 find dist/workspace-hxp -name "*.js" -size 0

```

Expected zip contents:

```

1 workspace-hxp.zip
2 |— index.html
3 |— main.[hash].js
4 |— polyfills.[hash].js
5 |— runtime.[hash].js
6 |— styles.[hash].css
7 |— assets/

```

```
8 |   images/
9 |   fonts/
10 |   i18n/
11 | app.config.json
```

Red flags to watch for:

- ❌ Missing `index.html`
- ❌ Empty JavaScript files
- ❌ Missing `app.config.json`
- ❌ No assets folder
- ❌ Zip file size < 5MB (likely incomplete)

3.2 Verify the Build

After the command completes, check that you have:

1. A **zip file** in the `dist` folder named `workspace-hxp.zip`
2. **No build errors** in the console
3. **All assets included** in the zip file

🔧 Step 4: Deploy in Studio Modeling

4.1 Navigate to Your Custom UI

1. **Go to Studio Modeling** and log in
2. **Find your Custom UI** in the project
3. **Click on it** to open the configuration

The screenshot shows the Studio Modeling interface for configuring a Custom UI. At the top, there's a breadcrumb navigation: Home > Example UI > Redirect_url. Below this is a toolbar with icons for cloud, checkmark, cloud, checkmark, eye, and a menu. The main content area is titled "Custom UI" and contains the following text: "The configuration of the custom UI is available during deployment." and "Hyland is not responsible for any security leaks your custom UI may cause, nor for its maintenance. [Learn more](#)". Below this is a form with two input fields: "Name" (with a red asterisk) containing "Redirect_url" and "Description". At the bottom, there's a section titled "Dummy-CustomUI" with a note "Only .zip files are accepted." and an "Upload" button with a download icon.

4.2 Upload the Package

1. **Click "Upload"** button

Custom UI

The configuration of the custom UI is available during deployment.

Hyland is not responsible for any security leaks your custom UI may cause, nor for its maintenance. [Learn more](#)

Name *

Redirect_url

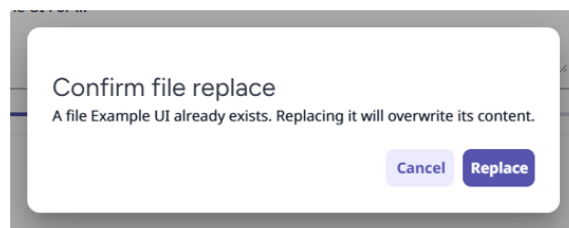
Description

Dummy-CustomUI

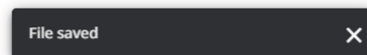
Only .zip files are accepted.

Upload

2. **Select the zip file** from your `dist` folder (`workspace-hxp.zip`)



3. **Wait for upload** to complete



4. **Verify the upload** was successful

4.3 Release and Deploy

1. **Release your project** to make the changes available
2. **Deploy to your environment** (staging or production)
3. **Verify deployment** was successful

🎉 Step 5: Test Your Deployed UI

5.1 Access Your Custom UI

1. **Navigate to your deployed app** in Hyland Experience
2. **Check that your custom UI loads** correctly
3. **Test all functionality** that you developed locally

5.2 Verify Everything Works

- **Navigation** works as expected

- **Custom pages** load properly
 - **Form widgets** function correctly
 - **Styling** looks correct
 - **No console errors** in the browser
-

Common Issues and Solutions

Build Fails with File Copy Errors

Problem: Build command fails with file copy errors.

Solution:

1. Make sure you're using the correct command syntax
2. Check that all file paths are correct
3. Ensure you have proper permissions to create/modify files

Upload Fails

Problem: Zip file won't upload to Studio Modeling.

Solution:

1. Check the file size (should be reasonable, not too large)
2. Verify the zip file isn't corrupted
3. Try uploading again or check your internet connection

UI Doesn't Load After Deployment

Problem: Custom UI shows errors or doesn't load.

Solution:

1. Check the browser console for errors
2. Verify all dependencies are included in the build
3. Make sure the configuration is correct in Studio Modeling

Missing Assets

Problem: Images, styles, or other assets don't load.

Solution:

1. Check that all assets are included in the build
 2. Verify file paths are correct
 3. Make sure assets are in the right location in the zip file
-

Best Practices

Before Packaging

- **Test thoroughly** in your local environment
- **Check for console errors** and fix them
- **Optimize images and assets** for production
- **Remove any debug code** or console.log statements

During Packaging

- **Use version control** to track changes
- **Test the build process** regularly
- **Keep build artifacts** for rollback if needed

After Deployment

- **Check for errors** in production logs
- **Gather user feedback** on the new UI

Advanced Packaging Options

Environment-Specific Builds

You can create different builds for different environments:

```
1 # Development build
2 npm run build workspace-hxp -- --configuration=development
3
4 # Production build
5 npm run build workspace-hxp -- --configuration=production
```

Custom Build Scripts

You can create custom build scripts in `package.json`:

```
1 {
2   "scripts": {
3     "build:staging": "nx build workspace-hxp --configuration=staging",
4     "package:staging": "npm run build:staging && tools/scripts/zip.mjs --source dist/workspace-hxp --target dist/workspace-hxp",
5   }
6 }
```

Next Steps

Now that your custom UI is deployed, you can:

- **Monitor its performance** in production
- **Gather user feedback** and make improvements
- **Plan future updates** and enhancements
- **Document the deployment process** for your team

Additional Resources

- **Hyland Experience Documentation:**
- **Angular Build Documentation:**
- **Nx Build Documentation:**

Questions?

Reach out to the team! We're here to help you! 

Thanks, and happy deploying! 

Related articles